

TLM: Deep Learning

Chapter 2: Regression and Classification

Outline:

- (1) What is regression?**
- (2) Prediction using linier regression**
- (3) Simple linier regression**
- (4) Multiple linier regression**
- (5) Simple linier regression using TensorFlow Keras**
- (6) Multiple and multivariate linear regression using the TensorFlow Keras API**
- (7) Classification tasks: use Logistic regression**

Dosen Penyusun:

David Agustriawan, S.Kom., M.S.c., Ph.D.

Ajie Kusuma Wardhana, S.Kom., M.Eng

What is regression?

Regression allows us to make a prediction from data by learning about relationship between a given set of dependent and independent variables. It can be used to draw a relationship between two or more things.

Consider the case of house price estimation. There are many factors that can have an impact of the house price: the number of rooms, the floor area, the locality, the availability of amenities, the parking space, and soon. Regression analysis can help us in finding the mathematical relationship between these factors and the house price.

Linier regression assumes a linier relationship between the input variable (X) and the output variable (Y). the basic idea of linier regression is building a model, using training data that can predict the output given the input such that the predicted output $\hat{Y}$ is near the observed training output Y for the input X. it involves finding a linier equation for the predicted value $\hat{Y}$ of the form:

$$\hat{Y} = W^T X + b$$

where $X = \{x_1, x_2, \dots, x_n\}$ are the n input variables, and $W = \{w_1, w_2, \dots, w_n\}$ are the linear coefficients, with b as the bias term. We can also expand the preceding equation to:

$$\hat{Y} = \sum_{i=1}^n x_i w_i + b$$

The bias term allows our regression model to provide an output even in the absence of any input; it provides us with an option to shift our data for a better fit. The error between the observed values (Y) and predicted values ($\hat{Y}$) for an input sample i is:

$$e_i = Y_i - \hat{Y}_i$$

The goal is to find the best estimates for the coefficients W and bias b , such that the error between the observed values Y and the predicted values $\hat{Y}$ is minimized. Let's go through some examples to better understand this.

Prediction using linier regression

Let us imagine a simpler world where only the area of the house (independent variable) determine the price (dependent variable). This case can be solved to use linier regression.

Simple linear regression

If we consider only one independent variable and one dependent variable, what we get is a simple linear regression. Consider the case of house price prediction, defined in the preceding section; the area of the house (A) is the independent variable, and the price (Y) of the house is the dependent variable. We want to find a linear relationship between predicted price $\hat{Y}$ and A , of the form:

$$\hat{Y} = A \cdot W + b$$

where b is the bias term. Thus, we need to determine W and b , such that the error between the price Y and the predicted price $\hat{Y}$ is minimized. The standard method used to estimate W and b is called the method of least squares, that is, we try to minimize the sum of the square of errors (S). For the preceding case, the expression becomes:

$$S(W, b) = \sum_{i=1}^N (Y_i - \hat{Y}_i)^2 = \sum_{i=1}^N (Y_i - A_i W - b)^2$$

We want to estimate the regression coefficients, W and b , such that S is minimized. We use the fact that the derivative of a function is 0 at its minima to get these two equations:

$$\frac{\partial S}{\partial W} = -2 \sum_{i=1}^N (Y_i - A_i W - b) A_i = 0$$

$$\frac{\partial S}{\partial b} = -2 \sum_{i=1}^N (Y_i - A_i W - b) = 0$$

These two equations can be solved to find the two unknowns. To do so, we first expand the summation in the second equation:

$$\sum_{i=1}^N Y_i - \sum_{i=1}^N A_i W - \sum_{i=1}^N b = 0$$

Take a look at the last term on the left-hand side; it just sums up a constant N time. Thus, we can rewrite it as:

$$\sum_{i=1}^N Y_i - W \sum_{i=1}^N A_i - Nb = 0$$

Reordering the terms, we get:

$$b = \frac{1}{N} \sum_{i=1}^N Y_i - \frac{W}{N} \sum_{i=1}^N A_i$$

The two terms on the right-hand side can be replaced by $\bar{Y}$, the average price (output), and $\bar{A}$, the average area (input), respectively, and thus we get:

$$b = \bar{Y} - W \bar{A}$$

In a similar fashion, we expand the partial differential equation of S with respect to weight W :

$$\sum_{i=1}^N (Y_i A_i - W A_i^2 - b A_i) = 0$$

Substitute the expression for the bias term b :

$$\sum_{i=1}^N (Y_i A_i - W A_i^2 - (\bar{Y} - W \bar{A}) A_i) = 0$$

Reordering the equation:

$$\sum_{i=1}^N (Y_i A_i - \bar{Y} A_i) - W \sum_{i=1}^N (A_i^2 - \bar{A} A_i) = 0$$

Playing around with the mean definition, we can get from this the value of weight W as:

$$W = \frac{\sum_{i=1}^N Y_i (A_i - \bar{A})}{\sum_{i=1}^N (A_i - \bar{A})^2}$$

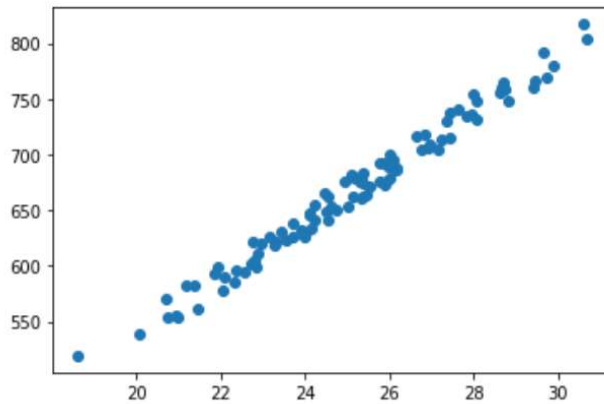
where $\bar{Y}$ and $\bar{A}$ are the average price and area, respectively. Let us try this on some simple sample data:

Simple linier regression with python only code:

https://github.com/PacktPublishing/Deep-Learning-with-TensorFlow-and-Keras-3rd-edition/blob/main/Chapter_2/simple_linear_regression.ipynb

1. Import the necessary modules. Using NumPy, pandas, and Matplotlib:
2. Generate random data with a linier relationship. In order to make it realistic, add a random noise element. Two variables (the cause, **area**, and the effect, **price**) follow a positive linier dependence:

```
#Generate a random data
np.random.seed(0)
area = 2.5 * np.random.randn(100) + 25
price = 25 * area + 5 + np.random.randint(20,50, size = len(area))
data = np.array([area, price])
data = pd.DataFrame(data = data.T, columns=['area', 'price'])
plt.scatter(data['area'], data['price'])
plt.show()
```



3. Calculate the two regression coefficients using the equations we defined

$$W = \frac{\sum(\text{price} * (\text{area} - \text{np.mean}(\text{area})))}{\sum((\text{area} - \text{np.mean}(\text{area}))^2)}$$

$$b = \text{np.mean}(\text{price}) - W * \text{np.mean}(\text{area})$$

```
print("The regression coefficients are", W,b)
```

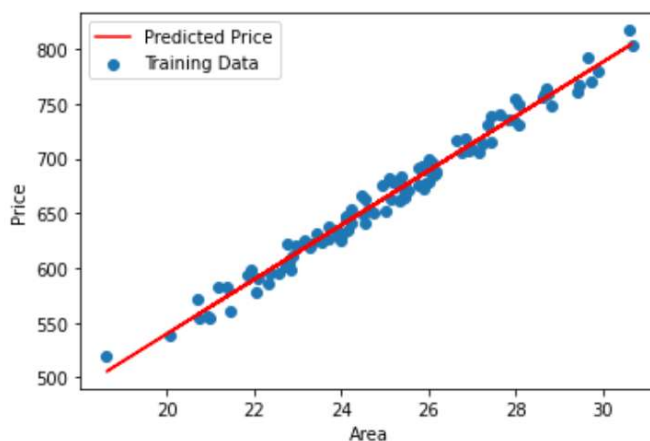
```
-----
The regression coefficients are 24.815544052284988 43.4989785533412
```

4. Predicting the new prices using the obtained weight and bias values

$$y_pred = W * \text{area} + b$$

5. Plot the predicted prices along the actual price. The predicted prices follow a linear relationship with the area:

```
plt.plot(area, y_pred, color='red', label='Predicted Price')
plt.scatter(data['area'], data['price'], label="Training Data")
plt.xlabel("Area")
plt.ylabel("Price")
plt.legend()
```



From Figure 2.2, we can see that the predicted values follow the same trend as the actual house prices.

Multiple linier regression

In most problems, the dependent variables depend upon multiple independent variables. Multiple linier regression finds a linier relationship between the many independent input variables (X) and the dependent output variable (Y), such that they satisfy the predicted Y value of the form:

$$\hat{Y} = W^T X + b$$

Where $X = \{X_1, X_2, \dots, X_n\}$ are the n independent input variables, and $W = \{W_1, W_2, \dots, W_n\}$ are the linier coefficients, with b as the bias term. The linier coefficients Ws are estimated using the method of least squares, that is, minimizing the sum of squared differences between predicted value ($\hat{Y}$) and observed value (Y). Thus, here try to minimize the loss function (also called squared error, and if we divide by n, it is the mean squared error):

$$\text{loss} = \sum_i (Y_i - \hat{Y}_i)^2$$

where the sum is over all the training samples.

Multivariate linier regression

There can be cases where the independent variables affect more than one dependent variable. For example, consider the case where we want to predict a **rocket's speed** and **its carbon dioxide emission**. These two will now be our dependent variables, and **both will be affected by the sensors reading the fuel amount, engine type, rocket body, and so on**. This is a case of multivariate linier regression.

Mathematically, a multivariate regression model can be depicted as below:

$$\hat{Y}_{ij} = w_{0j} + \sum_{k=1}^p w_{kj} x_{ik}$$

where $i \in [1, \dots, n]$ and $j \in [1, \dots, m]$. The term $\hat{Y}_{ij}$ represents the j^{th} predicted output value corresponding to the i^{th} input sample, w represents the regression coefficients, and x_{ik} is the k^{th} feature of the i^{th} input sample. The number of equations needed to solve in this case will now be $n \times m$. While we can solve these equations using matrices, the process will be computationally expensive as it will involve calculating the inverse and determinants. An easier way would be to use the gradient descent with the sum of least square error as the loss function and to use one of the many optimizers that the TensorFlow API includes.

In the next section, we will delve deeper into the TensorFlow Keras API, a versatile higher-level API to develop your model with ease.

Neural networks for linier regression

The simplest neural network, the simple perceptron, can be mathematically represented as:

$y = f(w^T x + b)$. where f is the activation function.

Simple linier regression using TensorFlow Keras

Code: https://github.com/PacktPublishing/Deep-Learning-with-TensorFlow-and-Keras-3rd-edition/blob/main/Chapter_2/simple_linear_regression_using_keras_API.ipynb

Predicting the price of a house given its area:

1. Import the necessary modules. Using NumPy, pandas, and Matplotlib:

```
import tensorflow as tf
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
import tensorflow.keras as K
from tensorflow.keras.layers import Dense
```
2. Generate random data with a linier relationship. In order to make it realistic, add a random noise element. Two variables (the cause, area, and the effect, price) follow a positive linier dependence:

```
#Generate a random data
np.random.seed(0)
area = 2.5 * np.random.randn(100) + 25
price = 25 * area + 5 + np.random.randint(20,50, size = len(area))
data = np.array([area, price])
data = pd.DataFrame(data = data.T, columns=['area', 'price'])
plt.scatter(data['area'], data['price'])
plt.show()
```
3. The input to neural network should be normalized; this is because input gets multiplied with weights, and if have very large numbers, the result of multiplication will be large, and soon our metrics may cross infinity (the largest number your computer can handle):

```
data = (data - data.min()) / (data.max() - data.min()) #normalize
```
4. Build the model; since it is simple linier repressor, we use a Dense layer with only one unit:

```
model = K.Sequential(
    Dense(1, input_shape = [1,], activation=None)
)
```



```
model.summary()
```

Model: "sequential"

Layer (type)	Output Shape	Param #
dense (Dense)	(None, 1)	2
Total params: 2		
Trainable params: 2		
Non-trainable params: 0		

- To train a model, we will need to define the loss function and optimizer. The loss function defines the quantity that our model tries to minimize, and the optimizer decides the minimization algorithm we are using. Additionally, we can also define metrics, which is the quantity we want to log as the model is trained. We define the loss function, optimizer and metrics using the compile function:

```
model.compile(loss='mean_squared_error', optimizer='sgd')
```

- Now, the model is defined, we just need to train it using the fit function. Observe that we are using a **batch_size of 32** and splitting the data into training and validation datasets using the `validation_split` argument of the fit function:

```
Model.fit(x=data['area'], y=data['price'], epochs=100, batch_size=32, verbose=1, validation_split=0.2)
```

```
Epoch 1/100
3/3 [=====] - 2s 150ms/step - loss: 0.0223 - val_loss: 0.0224
Epoch 2/100
3/3 [=====] - 0s 61ms/step - loss: 0.0197 - val_loss: 0.0196
Epoch 3/100
3/3 [=====] - 0s 40ms/step - loss: 0.0174 - val_loss: 0.0173
Epoch 4/100
3/3 [=====] - 0s 20ms/step - loss: 0.0155 - val_loss: 0.0153
Epoch 5/100
3/3 [=====] - 0s 25ms/step - loss: 0.0138 - val_loss: 0.0136
Epoch 6/100
3/3 [=====] - 0s 22ms/step - loss: 0.0124 - val_loss: 0.0122
Epoch 7/100
3/3 [=====] - 0s 27ms/step - loss: 0.0113 - val_loss: 0.0110
Epoch 8/100
3/3 [=====] - 0s 43ms/step - loss: 0.0102 - val_loss: 0.0099
Epoch 9/100
3/3 [=====] - 0s 46ms/step - loss: 0.0094 - val_loss: 0.0090
Epoch 10/100
3/3 [=====] - 0s 28ms/step - loss: 0.0087 - val_loss: 0.0083

Epoch 95/100
3/3 [=====] - 0s 14ms/step - loss: 0.0033 - val_loss: 0.0024
Epoch 96/100
3/3 [=====] - 0s 13ms/step - loss: 0.0033 - val_loss: 0.0024
Epoch 97/100
3/3 [=====] - 0s 13ms/step - loss: 0.0033 - val_loss: 0.0024
Epoch 98/100
3/3 [=====] - 0s 14ms/step - loss: 0.0033 - val_loss: 0.0024
Epoch 99/100
3/3 [=====] - 0s 19ms/step - loss: 0.0033 - val_loss: 0.0024
Epoch 100/100
3/3 [=====] - 0s 14ms/step - loss: 0.0032 - val_loss: 0.0024
```

- We have successfully trained a neural network to perform the task of linear regression. The mean squared error after training for 100 epochs is 0.0032 on training data and 0.0024 on validation data. We can get the predicted value for a given input using the predict function:

```
y_pred = model.predict(data['area'])
```

- Next, we plot a graph of the predicted and the actual data:

```
plt.plot(area, y_pred, color='red', label='Predicted Price')
plt.scatter(data['area'], data['price'], label="Training Data")
plt.xlabel("Area")
plt.ylabel("Price")
plt.legend()
```


9. Figure 2.3 shows the plot between the predicted data and the actual data. You can see that, just like the linear regressor, we have got a nice linear fit:

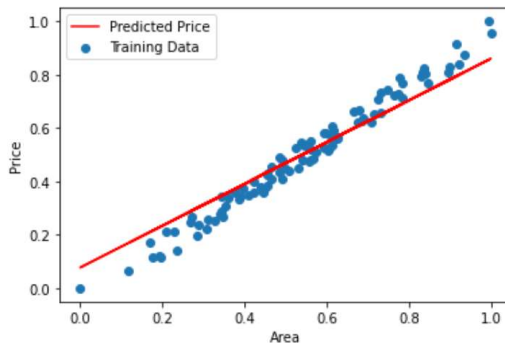


Figure 2.3: Predicted price vs actual price

10. In case you are interested in knowing the coefficient w and b , we can do it by printing the weights of the model using:
`model.weights`

```
[<tf.Variable 'dense/kernel:0' shape=(1, 1) dtype=float32,
 numpy=array([[ -0.33806288]], dtype=float32)>,
 <tf.Variable 'dense/bias:0' shape=(1,) dtype=float32,
 numpy=array([0.68142694], dtype=float32)>]
```

Multiple and multivariate linear regression using the TensorFlow Keras API

We can have more than one neural network layer, that is, we can build a deep neural network. A deep neural network is like applying multiple function approximators:

$$f(x) = f_L(f_{L-1}(\dots f_1(x)\dots))$$

with f_L being the function at layer L . from the expression above, we can see that if f was a linear function, adding multiple layers of neural network was not useful; however, using a non-linear activation function allows us to apply neural networks to the regression problems where dependent and independent variables are related in some non-linear fashion.

Below we will use a deep neural network, build using TensorFlow Keras to predict the fuel efficiency of a car, given its number of cylinders, displacement, acceleration, and so on. The data we use is available from UCI ML repository, the UCI repository of machine learning database (<http://cml.ics.uci.edu/>) or (<https://archive.ics.uci.edu/dataset/9/auto+mpg>).

Code: https://github.com/PacktPublishing/Deep-Learning-with-TensorFlow-and-Keras-3rd-edition/blob/main/Chapter_2/multiple_linear_regression_using_keras_API.ipynb

1. Start importing the modules that we will need. We will make use of the Keras Normalization layer. The Normalization layer shifts the data to a zero mean and one standard deviation. Also

since we have more than one independent variable, we will use Seaborn to visualize the relationship between different variables:

```
import tensorflow as tf
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
import tensorflow.keras as K
from tensorflow.keras.layers import Dense, Normalization
```

2. Let us first download the data from UCI ML repo

```
url = 'https://archive.ics.uci.edu/ml/machine-learning-databases/auto-mpg/auto-mpg.data'
```

```
column_names = ['mpg', 'cylinders', 'displacement', 'horsepower', 'weight', 'acceleration', 'model_year', 'origin']
```

```
data = pd.read_csv(url, names=column_names, na_values='?', comment='\t', sep=' ', skipinitialspace=True)
```

3. The data consists of eight features: mpg, cylinders, displacement, horsepower, weight, acceleration, model year, and origin. Though the origin of the vehicle can also affect the fuel efficiency "mpg" (miles per gallon), we use only seven features to predict the mpg value. Also, we drop any rows with NaN values:

```
data = data.drop('origin', 1)
print(data.isna().sum())
data = data.dropna()
```

4. Split the dataset into 80% of the 392 datapoints as training data and 20% as test dataset:

```
train_dataset = data.sample(frac=0.8, random_state=0)
test_dataset = data.drop(train_dataset.index)
```

5. Use Seaborn's pairplot to visualize the relationship between the different variables:

```
sns.pairplot(train_dataset[['mpg', 'cylinders', 'displacement', 'horsepower', 'weight', 'acceleration', 'model_year']], diag_kind='kde')
```

6. We can see that mpg (fuel efficiency) has dependencies on all the other variables, and the dependency relationship is non-linear, as none of the curves are linear:

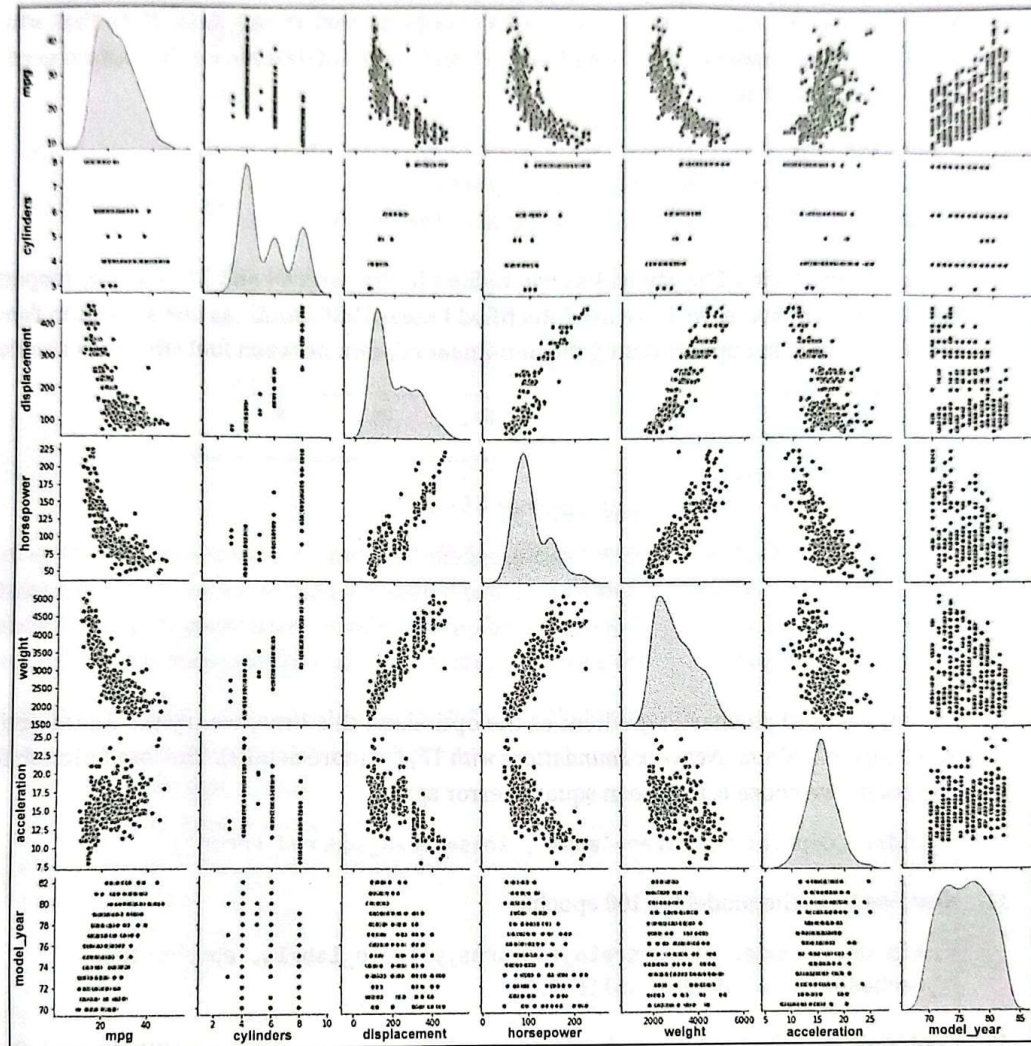


Figure 2.4: Relationship among different variables of auto-mpg data

- For convenience, separate the variables into input variables and the label that we want to predict:

```
train_features = train_dataset.copy()
test_features = test_dataset.copy()
```

```
train_labels = train_features.pop('mpg')
test_labels = test_features.pop('mpg')
```

- Use normalization layer of Keras to normalize the data. Note that while we normalized our inputs to a value with mean 0 and standard deviation 1, the output prediction 'mpg' remains as it is:

```
#Normalize
```

```
data_normalizer = Normalization(axis = 1)
```

```
data_normalizer.adapt(np.array(train_features))
```

9. Build the model with two hidden layers, with 64 and 32 neurons, respectively. for the hidden layers, we have used Rectified Linear Unit (ReLU) as activation function; this should help in approximating the non-linear relation between fuel efficiency and the rest of the variables:

```
model = K.Sequential([  
    data_normalizer,  
    Dense(64, activation='relu'),  
    Dense(32, activation='relu'),  
    Dense(1, activation=None)  
])  
model.summary()
```

10. Try to use ADAM optimizer and the loss function for the regression chose the mean squared error:

```
model.compile(optimizer='adam', loss='mean_squared_error')
```

11. Train the model for 100 epochs:

```
history = model.fit(x=train_features, y=train_labels, epochs = 100, verbose = 1, validation_split=0.2)
```

12. Check if our model is overfitted, underfitted, or properly fitted by plotting the loss curve. Both validation loss and training loss are near each other as we increase the training epochs; this indicate that the model is properly trained:

```
plt.plot(history.history['loss'], label='loss')  
plt.plot(history.history['val_loss'], label = 'val_loss')  
plt.xlabel('Epoch')  
plt.ylabel('Error [MPG]')  
plt.legend()  
plt.grid(True)
```

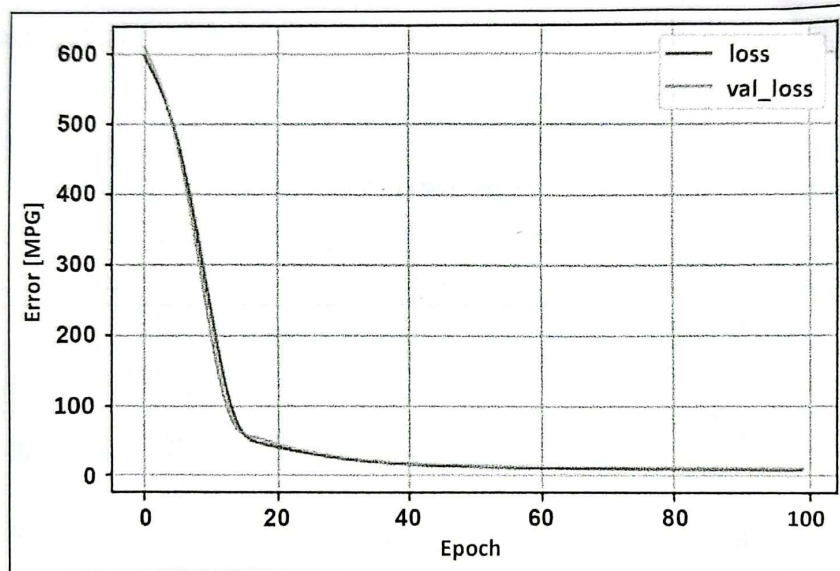


Figure 2.5: Model error

13. Compare the predicted fuel efficiency and the true fuel efficiency on the test dataset. Remember that the model has not seen a test

```

y_pred = model.predict(test_features).flatten()
a=plt.axes(aspect='equal')
plt.scatter(test_labels, y_pred)
plt.xlabel('True values [MPG]')
plt.ylabel('predictions [MPG]')
lims = [0, 50]
plt.xlim(lims)
plt.ylim(lims)
plt.plot(lims, lims)

```

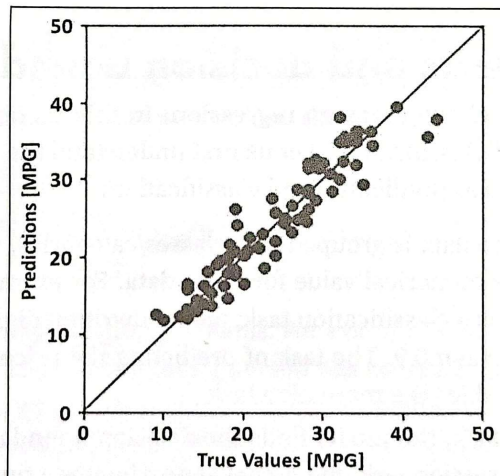


Figure 2.6: Plot between predicted fuel efficiency and actual values

14. Additionally, we can also plot the error between the predicted and true fuel efficiency:

```
error = y_pred - test_labels
```

```
plt.hist(error, bins=30)
```

```
plt.xlabel('Prediction Error [MPG]')
```

```
plt.ylabel('Count')
```

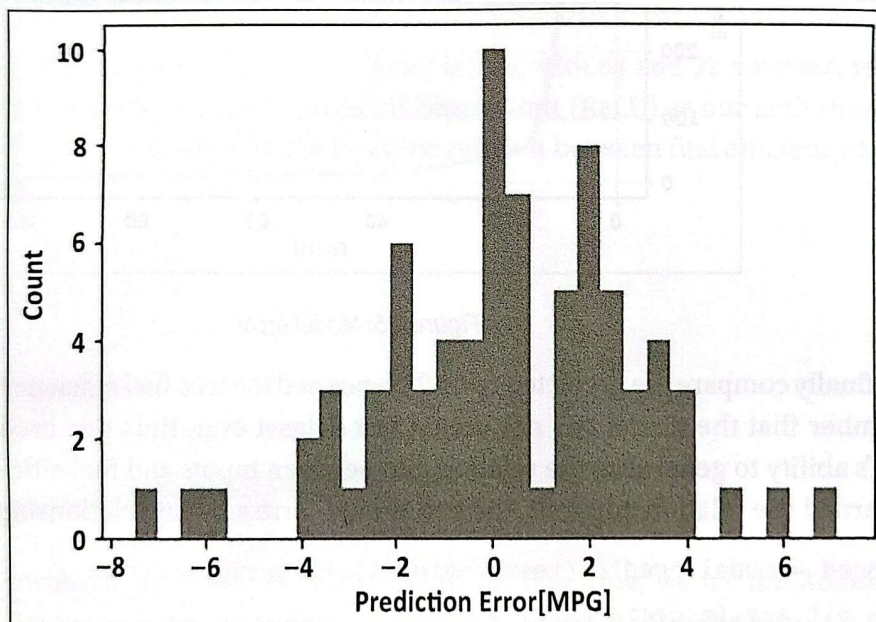


Figure 2.7: Prediction error

In case we want to make more than one prediction, that is, dealing with a multivariate regression problem, the only change would be that instead of one unit in the last dense layer, we will have as many units as the number of variables to be predicted.

Classification tasks: use Logistic regression

Logistic regression is used to determine the probability of an event. Conventionally, the event is represented as a categorical dependent variable.

Code link: https://github.com/PacktPublishing/Deep-Learning-with-TensorFlow-and-Keras-3rd-edition/blob/main/Chapter_2/logistic_regression_using_keras_API.ipynb

In order to differentiate between the network of simple linear regression, multiple linear regression and logistic regression we need to see the architecture of the network layer.